

Funciones en Python

Las funciones permiten organizar un programa en bloques reutilizables. Son una de las herramientas principales para escribir código claro, ordenado y fácil de mantener.

1. Qué es una función

Una función es un bloque de código con nombre que realiza una tarea concreta. En lugar de escribir las mismas instrucciones muchas veces, se agrupan en una función y se ejecutan cuando sea necesario.

Las funciones permiten dividir un programa en partes pequeñas. Cada parte resuelve un problema limitado: calcular una media, validar una contraseña, buscar un dato, convertir una medida o mostrar un mensaje.

Una función ayuda a que el programa sea más claro, porque el nombre de la función explica qué hace ese fragmento de código. También reduce errores, ya que una misma lógica se escribe una sola vez y puede reutilizarse.

En Python, las funciones se crean normalmente con la palabra reservada `def`. Después se escribe el nombre de la función, los paréntesis y dos puntos. El cuerpo de la función se escribe indentado.

```
def saludar():
    print("Hola")

saludar()
```

En este ejemplo, `saludar` es el nombre de la función. La función no se ejecuta al definirla, sino cuando se llama escribiendo `saludar()`.

La definición de una función responde a la pregunta: qué instrucciones quiero guardar. La llamada a una función responde a la pregunta: cuándo quiero ejecutar esas instrucciones.

2. Definir y llamar funciones

La estructura básica de una función en Python tiene tres partes: la palabra `def`, el nombre de la función y el bloque de instrucciones indentado.

El nombre debe ser claro y descriptivo. Es preferible usar nombres como `calcular_total`, `mostrar_menu` o `validar_edad` antes que nombres demasiado generales como `hacer`, `proceso` o `funcion1`.

La indentación forma parte de la sintaxis de Python. Todo lo que pertenece a la función debe estar desplazado hacia la derecha. Cuando termina la indentación, termina el cuerpo de la función.

```
def mostrar_datos():
    nombre = "Ana"
    edad = 20
    print("Nombre:", nombre)
    print("Edad:", edad)

mostrar_datos()
```

Primero se define la función. Después, fuera del bloque indentado, se llama a la función. Si se define una función pero nunca se llama, sus instrucciones no se ejecutan.

Una función puede llamarse muchas veces. Cada llamada ejecuta de nuevo el bloque completo.

3. Parámetros y argumentos

Los parámetros son nombres que aparecen en la definición de una función. Sirven para que la función pueda recibir datos desde fuera.

Los argumentos son los valores concretos que se envían a la función cuando se llama. En la práctica, el parámetro es la variable que espera la función y el argumento es el dato que se entrega.

Una misma función puede funcionar con datos distintos si recibe parámetros. Esto evita crear muchas funciones casi iguales.

```
def saludar(nombre):  
    print("Hola,", nombre)  
  
saludar("Ana")  
saludar("Luis")
```

En la definición, nombre es el parámetro. En las llamadas, Ana y Luis son argumentos.

Las funciones pueden tener varios parámetros. El orden importa cuando se usan argumentos posicionales.

4. Argumentos posicionales y argumentos con nombre

Cuando se llama a una función, los argumentos pueden pasarse por posición o por nombre.

En los argumentos posicionales, Python asigna cada valor al parámetro correspondiente según el orden en que aparecen.

En los argumentos con nombre, se indica explícitamente qué parámetro recibe cada valor. Esto mejora la claridad cuando la función tiene varios datos.

```
def crear_ficha(nombre, edad, curso):  
    print(nombre, "-", edad, "-", curso)  
  
crear_ficha("Ana", 20, "Python")  
crear_ficha(edad=21, curso="Programación", nombre="Luis")
```

La primera llamada usa argumentos posicionales. La segunda llamada usa argumentos con nombre, por eso el orden ya no es tan importante.

Los argumentos con nombre son útiles cuando hay varios parámetros del mismo tipo, por ejemplo varios números o varios textos.

5. return y print

print muestra información por pantalla. return devuelve un resultado al lugar donde se llamó la función.

Esta diferencia es fundamental. Una función puede imprimir un dato, pero si no usa return, ese dato no queda disponible para seguir calculando con él.

Cuando una función llega a return, termina su ejecución y entrega el valor indicado. El valor devuelto puede guardarse en una variable, usarse en una operación o pasarse a otra función.

```
def sumar(a, b):  
    resultado = a + b  
    return resultado  
  
total = sumar(4, 6)  
print(total)
```

La función sumar devuelve el resultado. Después, ese resultado se guarda en la variable total.

Si una función no tiene return, Python devuelve automáticamente None. None representa ausencia de valor.

6. Funciones sin return y funciones con return

No todas las funciones necesitan devolver un valor. Algunas funciones solo realizan una acción, como mostrar un menú o imprimir un informe.

Cuando una función calcula algo que se va a utilizar después, normalmente debe tener return.

Una forma sencilla de decidirlo es preguntar: necesito conservar el resultado para usarlo en otra parte del programa. Si la respuesta es sí, conviene usar return.

```
def mostrar_bienvenida():
    print("Bienvenido al programa")

def calcular_iva(precio):
    return precio * 0.21

mostrar_bienvenida()
iva = calcular_iva(100)
print("IVA:", iva)
```

mostrar_bienvenida no devuelve nada porque solo muestra un mensaje. calcular_iva sí devuelve un número porque ese resultado puede utilizarse después.

En programas pequeños puede parecer que print y return son parecidos, pero en programas reales cumplen funciones distintas.

7. Ámbito de las variables

El ámbito de una variable indica en qué parte del programa puede utilizarse. Una variable creada dentro de una función es local: existe dentro de esa función.

Una variable creada fuera de las funciones es global: puede leerse desde distintas partes del programa. Aun así, modificar variables globales desde dentro de funciones puede hacer que el programa sea más difícil de entender.

Como regla general, una función debe recibir los datos que necesita mediante parámetros y devolver resultados mediante return.

```
def calcular_doble(numero):
    doble = numero * 2
    return doble

resultado = calcular_doble(5)
print(resultado)
```

La variable doble solo existe dentro de la función calcular_doble. Fuera de la función, el valor se conserva gracias al return y a la variable resultado.

Este modo de trabajar hace que las funciones sean más independientes y más fáciles de revisar.

8. Valores por defecto

Un parámetro puede tener un valor por defecto. Esto significa que, si al llamar a la función no se envía un argumento para ese parámetro, Python usará el valor indicado en la definición.

Los valores por defecto permiten crear funciones flexibles. Algunos datos serán obligatorios y otros opcionales.

Los parámetros con valor por defecto deben colocarse después de los parámetros obligatorios.

```
def calcular_precio(precio, descuento=0):
    precio_final = precio - precio * descuento / 100
    return precio_final

print(calcular_precio(100))
print(calcular_precio(100, 15))
```

La primera llamada no indica descuento, por eso se usa 0. La segunda llamada indica 15, por eso se aplica un descuento del 15%.

Los valores por defecto hacen que una función pueda utilizarse en casos simples y en casos más completos.

9. Varios valores de retorno

Una función puede devolver más de un valor. En Python, cuando se devuelven varios valores separados por comas, realmente se devuelve una tupla.

Después, esos valores pueden guardarse en una sola variable o desempaquetarse en varias variables.

Esto resulta útil cuando una función calcula varios datos relacionados, por ejemplo una media, un máximo y un mínimo.

```
def analizar_notas(notas):
    media = sum(notas) / len(notas)
    nota_maxima = max(notas)
    nota_minima = min(notas)
    return media, nota_maxima, nota_minima

media, maxima, minima = analizar_notas([6, 8, 5, 9])
print(media, maxima, minima)
```

La función devuelve tres valores. En la llamada, esos tres valores se guardan directamente en tres variables.

Aunque este mecanismo es cómodo, no conviene devolver demasiados valores a la vez. Si hay muchos datos, puede ser más claro usar un diccionario.

10. *args y **kwargs

*args permite que una función reciba un número variable de argumentos posicionales. Dentro de la función, args se comporta como una tupla.

**kwargs permite recibir un número variable de argumentos con nombre. Dentro de la función, kwargs se comporta como un diccionario.

Estos mecanismos se usan cuando no se sabe de antemano cuántos datos recibirá la función. No son necesarios en todas las funciones, pero conviene conocerlos.

```
def sumar_todo(*numeros):
    total = 0
    for numero in numeros:
        total += numero
    return total

def mostrar_datos(**datos):
```

```

    for clave, valor in datos.items():
        print(clave, ":", valor)

print(sumar_todo(2, 4, 6, 8))
mostrar_datos(nombre="Ana", edad=20, curso="Python")

```

En `sumar_todo`, todos los números llegan agrupados en `numeros`. En `mostrar_datos`, los datos con nombre llegan como pares clave-valor.

Aunque se escriben normalmente como `args` y `kwargs`, lo importante son los símbolos `*` y `**`.

11. Anotaciones de tipo y docstrings

Python permite añadir anotaciones de tipo a los parámetros y al valor de retorno. Estas anotaciones no convierten Python en un lenguaje de tipado estático, pero ayudan a leer el código.

Una anotación indica qué tipo de dato se espera. Por ejemplo, `nombre: str` indica que se espera un texto. -> `float` indica que la función devuelve un número decimal.

Un docstring es una cadena de texto situada al principio de una función. Sirve para documentar qué hace la función.

```

def calcular_media(notas: list[float]) -> float:
    """Calcula la media de una lista de notas."""
    total = sum(notas)
    return total / len(notas)

media = calcular_media([7.5, 8.0, 6.5])
print(media)

```

Las anotaciones no sustituyen a la lógica del programa. Si se pasan datos incorrectos, Python no siempre lo impedirá automáticamente.

Aun así, las anotaciones y los docstrings ayudan a que el código sea más comprensible.

12. Funciones lambda

Una función lambda es una función pequeña y anónima. Se llama anónima porque no se define con `def` ni necesita un nombre obligatorio.

La sintaxis básica es: `lambda parámetros: expresión`. La expresión se evalúa y su resultado se devuelve automáticamente.

Las funciones lambda se usan sobre todo cuando se necesita una función sencilla durante poco tiempo, por ejemplo para ordenar datos o transformar valores.

```

doble = lambda numero: numero * 2
print(doble(5))

personas = [
    {"nombre": "Ana", "edad": 23},
    {"nombre": "Luis", "edad": 19},
    {"nombre": "Marta", "edad": 31}
]

ordenadas = sorted(personas, key=lambda persona: persona["edad"])
print(ordenadas)

```

La primera lambda calcula el doble de un número. En el segundo ejemplo, la lambda indica a `sorted` que debe ordenar la lista usando la edad de cada persona.

Si la lógica es larga o necesita varias líneas, es mejor usar `def`. Lambda debe reservarse para expresiones simples.

13. Funciones como objetos

En Python, las funciones son objetos. Esto significa que pueden guardarse en variables, pasarse como argumentos y devolverse desde otras funciones.

Esta característica permite crear programas más flexibles. Por ejemplo, una función puede recibir otra función para decidir cómo procesar una lista de datos.

Algunas funciones incorporadas, como `map`, `filter` o `sorted`, aceptan funciones como argumento.

```
def cuadrado(numero):
    return numero * numero

numeros = [1, 2, 3, 4]
cuadrados = list(map(cuadrado, numeros))
pares = list(filter(lambda numero: numero % 2 == 0, numeros))

print(cuadrados)
print(pares)
```

`map` aplica una función a cada elemento de una colección. `filter` conserva solo los elementos que cumplen una condición.

En muchos casos, un bucle `for` puede ser más claro para principiantes. `map` y `filter` son útiles, pero no obligatorios para resolver todos los problemas.

14. Recursividad

Una función recursiva es una función que se llama a sí misma. Se utiliza para resolver problemas que pueden dividirse en versiones más pequeñas del mismo problema.

Toda función recursiva necesita un caso base. El caso base es la condición que detiene las llamadas. Sin caso base, la función se llamaría indefinidamente hasta producir un error.

La recursividad puede ser elegante, pero no siempre es la solución más sencilla. Para muchos problemas iniciales, un bucle `while` o `for` resulta más fácil de entender.

```
def factorial(numero):
    if numero == 0:
        return 1
    else:
        return numero * factorial(numero - 1)

print(factorial(5))
```

El factorial de 5 se calcula como $5 * 4 * 3 * 2 * 1$. La función se llama a sí misma con un número cada vez menor hasta llegar a 0.

El caso base es `numero == 0`. En ese momento la función devuelve 1 y empiezan a resolverse las llamadas pendientes.

15. Organización y buenas prácticas

Una función debe tener una responsabilidad clara. Si una función hace demasiadas cosas, suele ser mejor dividirla en varias funciones más pequeñas.

Los nombres deben explicar la acción que realiza la función. En Python es habitual escribir los nombres en minúscula y separar palabras con guion bajo: `calcular_total`, `validar_usuario`, `mostrar_resultado`.

Conviene evitar funciones excesivamente largas. Si cuesta explicar qué hace una función en una frase, probablemente está mezclando demasiadas tareas.

También es importante distinguir entre calcular y mostrar. Una función que calcula un resultado suele devolverlo con `return`. Una función que muestra información puede usar `print`.

```
def calcular_total(precios):
    total = 0
    for precio in precios:
        total += precio
    return total

def aplicar_descuento(total, porcentaje):
    return total - total * porcentaje / 100

def mostrar_resultado(total_final):
    print("Total final:", total_final)

precios = [10, 20, 15]
total = calcular_total(precios)
total_final = aplicar_descuento(total, 10)
mostrar_resultado(total_final)
```

El ejemplo separa el programa en tres funciones: una calcula el total, otra aplica el descuento y otra muestra el resultado.

Esta forma de organizar el código facilita la lectura, la corrección de errores y la ampliación del programa.